DRAFT: Open Pool Questions

Advanced Algorithms

At least 30% of the final (and midterm) exam questions will be drawn **verbatim** from this pool.

Lecture 1: Stable Marriage

Question 1. *Stable Matchings*

This set of questions captures the learning goals of the first lecture on the stable marriage problem.

1. Define an Euler Tour and show that every connected graph with all even degrees must have an Euler Tour.
2. Define the term stable marriage problem, blocking pair and stable matching.
3. Define the Greedy Algorithm as in the lecture and show that it might run forever.
4. Explain the Gale Shapeley algorithm and show that it returns a stable matching.
5. Define a men optimal stable matching. and show that the Gale Shapeley algorithm returns a men optimal stable matching.

Lecture 2: Min Flow Max Cut

Question 2. *Minimum Vertex Cut*

Show that determining the maximum flow in a network with arc and vertex capacity constraints can be reduced to an ordinary maximum-flow problem on a flow network of comparable size.

Question 3. *Escaping a Grid*

An $n \times n$ grid is an undirected graph consisting of n rows and n columns of vertices as shown in the figure. We denote the vertex in the ith row and jth column by $v_{i,j}$. All vertices in a grid have exactly four neighbours, except for the *boundary* points, which are the vertices $v_{i,j}$ with $i = 1$, $i = n$, $j = 1$ or $j = n$.

Given $m \leq n^2$ starting points $v_{x_1,y_1}, \ldots, v_{x_m,y_m}$ in the grid, the *grid escape problem* is to determine if there are m vertex disjoint paths from the starting points to any m different points in the boundary.

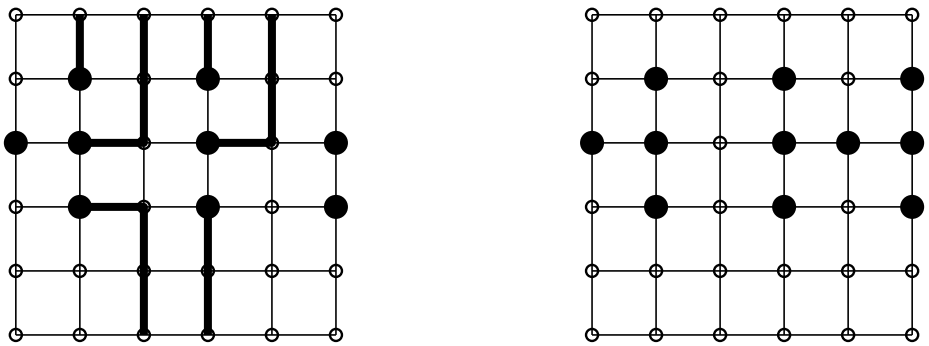

Figure 1: Grids for the grid escape problem. Starting points are the larger black disks; other grid vertices are smaller circles. The first grid is shown with a solution the the grid escape problem; the second has no solution

Describe an algorithm with running time polynomial in n that solves the escape problem and analyse its running time. (Be careful that n and m describe the size of the grid and number of starting points, not the number of vertices and arcs as usual).

Lecture 3: Min Flow Max Cut

Question 4. *Phasing out capital equipment*

A shipping company wants to phase out a fleet of (homogeneous) general cargo ships over a period of p years. Its objective is to maximize its cash assets at the end of the p years by considering the possibility to prematurely selling ships and temporary replacing them by ships they rent.

The company faces a non-increasing demand for ships: every year, the number of ships needed is at most the number needed the previous year. Let $d(i)$ be the number of ships needed in year i. The company can possibly have more than $d(i)$ ships in year i, but may not have less than these.

If a ship is sold in year i, this yields s_i euros. Each ship the company has in year i gives r_i euros as profit. If the company has less than $d(i)$ ships in year i, it must hire additional ships. This costs h_i in year i. Assume that $h_i > r_i$ for each i.

The company wants to meet its commitments (have sufficient ships either owned or rented each year), and have as much cash assets as possible at the end of the pth year.

Formulate this problem as a minimum cost flow problem.

(Exercise 9.6 from Network Flows, Ahuja, Magnanti, Orlin.)

Lecture 4: Planar Graphs

Question 5. *Planar Graphs*

This set of questions captures the learning goals of the lecture on the planar graphs. We assume that G is a planar graph on n vertices and m edges.

1. Give and proof the Euler Formula.
2. Show that $m \leq 3n - 6$ and the average degree below 6.
3. Define graph minors and state the Kuratowski theorem.
4. Show that planar graphs are 5-colorable.
5. Show that K_{33} is non-planar in an elementary way.
6. State the Planar Separator Theorem.
7. Show that 3-coloring can be solved in $2^{O(\sqrt{n})}$ time on planar graphs.
8. Show that 3-coloring is NP-hard for planar graphs.

Lecture 5: Real Models of Computation

Question 6. *Real Models of Computation*

This set of questions captures the learning goals of the lecture on real models of computation.

1. Explain a program on a word RAM. (Not a formal definition.)
2. Explain a Turing machine. (Not a formal definition.)
3. Give a comparison of advantages and disadvantages of the two models.
4. Explain what a program on a real RAM is and give both a motivation and concerns about this model of computation.
5. Show how to compute $k!$ in $O(\log^2 k)$ time on a real RAM that is allowed to use rounding.
6. Show how to compute the greatest common divisor $\gcd(a, b)$ of two numbers in $O(\min \log a, \log b)$ time on a real RAM that is allowed to use rounding.
7. Show that we can find a factor of a number in polynomial time on the real RAM that is allowed to use rounding. You are allowed to use the the answer of the previous two questions.
8. Define a straight line program and PosSLP.
9. Show that $\mathrm{P}^{\mathrm{PosSLP}} = \mathrm{P}\mathbb{R}$.

Lecture 6: Matching

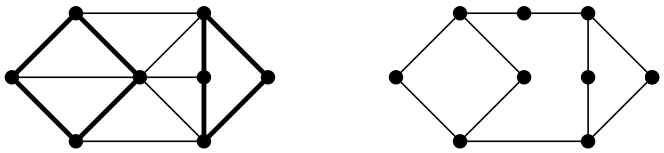

Figure 2: A graph with a cycle cover and a graph without a cycle cover

Question 7.

Consider the following problem:

> Partition into Cycles
> **Instance:** Undirected graph $G = (V, E)$.
> **Question:** Can we partition G into cycles, i.e., is there a collection of simple cycles $C_1, \ldots C_r$ in G, such that each vertex belongs to exactly one of the cycles in the collection?

Give a polynomial time algorithm that solves this problem.

Lecture 7: Existential Theory of the Reals

Question 8. *Existential Theory of the Reals*

This set of questions captures the learning goals of the first lecture on the existential theory of the reals.

1. Give a definition of the algorithmic problem ETR and the complexity class $\exists\mathbb{R}$ based on this definition.
2. Give the machine model description of $\exists\mathbb{R}$ and a sketch of a proof that it is equivalent with the logic based definition.
3. Show that NP $\subseteq \exists\mathbb{R}$.
4. Define the order types and the order type problem. Give a motivation to study order types.
5. Define ETR-AM as in the lecture and show that it is $\exists\mathbb{R}$-complete.
6. Define the Partial Order Type Problem. Suppose that we can use order-types to also enforce pairs of lines, defined by two points each, to be parallel. Show that the Partial Order Type Problem is $\exists\mathbb{R}$-complete.

Lecture 8: Exact Exponential-Time Algorithms

Question 9. *Exact Algorithm for Exact 3-SAT*

Suppose we have a set of Boolean variables $x_1, \ldots, x_n$, a set of clauses, where each clause is a subset of $\{x_1, \ldots, x_n, \text{not}(x_1), \ldots, \text{not}(x_n)\}$, and each clause has at most three literals (elements). In the Exact 3-Satisfiability, we want to determine if there exists a truth assignment such that each clause has *exactly* one true literal.

For instance, if we have clauses $\{x_1, x_2, x_3\}$, $\{$ not(x_1), x_2, $x_4\}$, then setting x_1 to true, x_2 to false, x_3 to false, and x_4 to true is a solution. Also, setting x_1 and x_2 to true does not lead to a solution, as the first clause has two literals that are true.

1. Show that we can create a reduction rule using polynomial time that transforms an instance of Exact 3-Satisfiability into an equivalent smaller instance of Exact 3-Satisfiability where all clauses have size two or three. (Smaller means, not increasing the number of variables or clauses)

2. Show that we can create a reductio rule using polynomial time that transforms an instance of Exact 3-Satisfiability into an equivalent smaller instance of Exact 3-Satisfiability where all clauses have size exactly three. (Smaller means, not increasing the number of variables or clauses)

3. Give an exponential-time algorithm that solves the Exact 3-satisfiability problem. Analyse the time of your algorithm. Your algorithm should use $\mathcal{O}^*(c^n)$ time for some $c < 2$.

Question 10. *Bisection Width*

Consider the following problem.

> **Given:** Graph $G = (V, E)$, with $|V|$ even.
> **Question:** Find a partition of V into two sets V_1, V_2, with $|V_1| = |V_2| = |V|/2$, such that the number of edges with one endpoint in V_1 and one endpoint in V_2 is as small as possible.

This problem is NP-complete, and an $\mathcal{O}^*(2^n)$ algorithm is trivial. Find an efficient exact algorithm for this problem with running time $\mathcal{O}^*(c^n)$ for some $c < 2$.

Lecture 9: Existential Theory of the Reals

Question 11. *Exact Algorithm for Exact 3-SAT*

1. Describe point-line duality.

2. Show the above-below property of point-line duality.

3. Define the cyclic order of a point with respect to a point set. Describe what the cyclic order corresponds to in the dual.

4. Define geometric intersection graphs and unit disk graphs in particular.

5. Show that deciding whether a given graph is a unit disk intersection graph is $\exists\mathbb{R}$-complete.

6. Show that Optimal Curve Straightening problem is $\exists\mathbb{R}$-complete.

Lecture 10: Inclusion/Exclusion & Approximation

Question 12. *Inclusion/exclusion for set cover*

Let U be any set of size m ($m = |U|$), and let $\mathcal{F}$ be a set of subsets of U. A set cover $\mathcal{C} \subset \mathcal{F}$ is a set of sets that together cover all sets in U, i.e., $\bigcup_{S \in \mathcal{C}} S = U$. A minimum set cover is a set cover using the smallest number of sets from $\mathcal{F}$ as possible.

Give an $\mathcal{O}^*(2^m)$ time and polynomial-space algorithm that counts the number of minimum set covers of $\mathcal{F}$.

Question 13. *Approximation non-symmetric TSP*

Suppose we have an instance of Travelling Salesman Problem that is not symmetric but still fulfils the triangle inequality. I.e., batery usage by an electric car in an mountainous area. We know that for every two cities i and j, we have that $d(i, j) \leq 2d(j, i)$, and of course, $d(j, i) \leq 2d(i, j)$.

Give a c-approximation algorithm for this problem that runs in polynomial time.

Lecture 11: Approximation Algorithms II

Question 14. *Approximating Steiner Tree*

Let $G = (V, E)$ be an undirected connected graph, let $T \subseteq V$ be set of vertices called the *terminals*, and let $N = V \setminus T$ be the non-terminal vertices. In the Steiner Tree problem, we are asked to compute a minimum-size set of edges $S \subseteq E$ such that S connects all vertices in T. That is, if we let $V(S)$ be the set of endpoints of edges in S, then $(V(S), S)$ is a tree with $T \subseteq V(S) \subseteq V$ connecting the vertices in T, where it is allowed to use vertices from N as intermediate vertices in the tree.

1. Prove that the procedure below is a 2-approximation to Steiner Tree.

   Define the complete graph $G_T = (T, F)$ to be a graph on the terminal vertices T, with $F = \{\{u, v\} : u \in T, v \in T, u \neq v\}$ and cost function $c : F \to \mathbb{N}$, where the cost of edge $\{t_1, t_2\} \in F$ equals the length of the shortest path from t_1 to t_2 in the original graph G. Also, define $P(\{t_1, t_2\})$ be the set of edges underlying such a shortest path from t_1 to t_2 in G.

   Consider the following algorithm for Steiner tree: given $G = (V, E)$ and T, first compute G_T and a minimum spanning tree $M \subseteq F$ on G_T w.r.t. to the cost function c, and then output the edge set:

   $$\bigcup_{m \in M} P(m)$$

   Note that a minimum spanning tree is a minimum cost edge set connecting *all* vertices in the graph.

2. Prove that there is no FPTAS for Steiner tree unless $P = NP$.

   You do not need part 1 to prove this, and partial points will be awarded if you prove the weaker statement that no additive-constant-factor approximation exists unless $P = NP$.

Lecture 12: Parametrized Complexity I

Question 15.

1. Define a parameterized problem, FPT, XP, and para-NP-hard.
2. Describe the idea of a branching algorithm in general.
3. Define Vertex Cover and show that Vertex Cover can be solved in FPT time using a branching algorithm.
4. Define Cluster Editing and give a branching algorithm for it.
5. Give one parameterized problem that is para-NP-complete.
6. Define the notion of a kernel.
7. Show that a problem is FPT if and only if it has a kernel.
8. Show that Vertex Cover has a kernel of size $O(k^2)$.
9. Define Convex String Recoloring and show that it has a kernel of size $O(k^2)$.
10. Define the Directed k-Path problem and acyclic graphs. Show that Directed k-Path can be solved in polynomial time on acyclic graphs.
11. Explain what a constant one-sided error algorithm is. Show that there is a constant one-sided error algorithm for Directed k-Path with running time $O(k^k \cdot n^c$, for some suitable constant c.

Lecture 13: Treewidth

Question 16. *Treewidth of grid graphs*

The p by q grid graph $GR_{p \times q}$ is the graph $(V_{p \times q}, E_{p \times q})$, with $V_{p \times q} = \{v_{i,j} \mid 1 \leq i \leq p \wedge 1 \leq j \leq q\}$, and $E_{p \times q} = \{\{v_{i,j}, v_{i+1,j}\} \mid 1 \leq i < p \wedge 1 \leq j \leq q\} \cup \{\{v_{i,j}, v_{i+1,j}\} \mid 1 \leq i < p \wedge 1 \leq j \leq q\}$.

Show, by giving a construction of a tree decomposition, that the treewidth of a p by q grid graph with $p \leq q$ is at most p.

Question 17. *Treewidth and cliques*

A clique in a graph $G = (V, E)$ is a subset of the vertices $C \subseteq V$ such that for any pair of vertices $c_1, c_2 \in C$ their is an edge between the vertices in G: $\{c_1, c_2\} \in E$.

Show that for any tree decomposition T of G, T must have a bag X_i for which $C \subseteq X_i$. Conclude that the treewidth of G is a least $|C| - 1$.

Lecture 14: Complexity for Approximation and FPT

Question 18. *L-reduction for Maximum Cut on Multigraphs*

We consider the approximation complexity of the Max Cut on Multigraphs problem and the Max Not-All-Equal 3-SAT problem.

A multigraph $G = (V, E)$ is a graph that can contain multiple identical edges. Given a multigraph $G = (V, E)$, a cut (V_1, V_2) in G is a partitioning of the vertices V into two sets V_1, V_2 ($V_1 \cap V_2 = \emptyset$, $V_1 \cup V_2 = V$). The value of the cut (V_1, V_2) is the number of edges $(u, v) \in E$ with one endpoint in V_1 and one endpoint in V_2. In the Max Cut on Multigraphs problem, we are given a multigraph and we are asked to compute a cut with maximum value. See Figure 3 for an example.

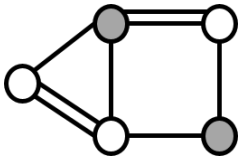

Figure 3: Example of a multigraph with a cut defined by the white and the grey vertices. This is not a cut of maximum value.

In the Max Not-All-Equal 3-SAT problem, we are given a set of variables X and a set of clauses $\mathcal{C}$ over the variables X (clauses that can have literals x and $\neg x$ for variables $x \in X$), where each clause consists of *exactly two or three literals*. In this problem, a clause $C \in \mathcal{C}$ is satisfied by a truth assignment ϕ, if at least one literal in C is made true by ϕ and at least one literal in C is made false by ϕ. The question in this problem is to compute a truth assignment that satisfies the maximum number of clauses. In this question you may use the fact that Max Not-All-Equal 3-SAT is APX-complete.

Prove that Max Cut on Multigraphs *is APX-hard by giving an L-reduction from* Max Not-All-Equal 3-SAT .

Lecture 15: Kernelization and Randomized Algorithms

Question 19. *Vertex Cover Kernel*

- Define Integer Linear Programming and show how Vertex Cover can be encoded as an Integer Linear Programming.
- Show that Vertex Cover has a linear sized kernel. (If this question comes up in the exam, we will ask for only part of the proof.)
- Define 2 list coloring and show that it can be solved in polynomial time.
- Using the answer to the previous question describe a randomized constant one-sided error for 3-coloring a graph with runtim $O(1.5^n)$.